

CSC 6991 Final Report – Quinn Aho

QEM Studio is a desktop tool I built for simplifying 3D triangle meshes, reducing the number of triangles in a model while keeping its shape. It uses the Quadric Error Metrics (QEM) algorithm, with three added modes for protecting important features: open boundaries, sharp creases, and curved regions. Everything is wrapped in a graphical interface that shows the mesh live, previews what will be protected before you simplify, and lets you compare the before and after side by side. The tool runs on its own, no dependency on the larger engine it started inside of.

A single 3D model can have hundreds of thousands of triangles. That's too many for most real-world uses: real-time rendering, ray tracing, streaming assets over the network, and simulation pipelines all need lighter versions of the same shape. The goal of simplification is simple in principle, fewer triangles, same silhouette, but doing it without destroying the features that make the shape recognizable is the hard part.

QEM collapses one edge of the mesh at a time, always choosing the cheapest collapse first. Each vertex carries a small matrix (a quadric) that tracks how far it has drifted from the original surface. Collapses in flat, featureless regions cost almost nothing and happen quickly; collapses in detailed regions cost a lot and are avoided. The algorithm keeps going until the mesh reaches a target number of triangles.

On top of the baseline algorithm I added three preservation modes. Boundary preservation protects open edges of the mesh so holes don't close up. Sharp-edge preservation protects hard creases — the kind you see on the edge of a cube, so they don't get rounded off. Curvature preservation identifies curved regions and tells the algorithm to collapse flat regions first, sparing the detail. Each mode works by making certain collapses more expensive, and for boundaries and creases I also added a strict validation layer that flat-out refuses the collapses that would destroy the feature.

Around the algorithm I built a standalone graphical tool. It has an orbit camera with solid, wireframe, or combined display modes. As you drag the detection sliders, overlays appear live in the viewport: red lines for boundaries, orange for sharp edges, yellow dots for the most curved vertices. Simplification runs on a background thread so the window doesn't freeze on large meshes, and a split-viewport compare mode loads the simplified result alongside the original so you can see exactly what changed. Finished meshes can be saved as OBJ files.

The biggest performance win was updating only the local neighborhood after each edge collapse, rather than recomputing everything from scratch. On the dragon model (870,000 triangles), this is the difference between a tool you can actually use and one that hangs forever.

The preservation modes hold up on the kinds of meshes they were designed for. The cow model simplified to 10% keeps its horns, eye sockets, and hooves sharp with all three flags on, where plain QEM blurs them into rounded blobs. An architectural scene at 70% stays recognizable as a building with preservation on, and turns into unreadable triangle soup without it.

The interface paid off more than I expected. Being able to drag a slider and see the detection overlay update immediately made it much easier to understand what the algorithm was actually doing, more useful than any static screenshot. The split-viewport compare mode turned out to be the single most valuable feature for showing what preservation does.

Curvature preservation under-detects on dense meshes. On the cow model the curvature signal is clean and the preservation visibly helps. On the bunny model (about ten times more triangles), the same metric barely registers the obviously-curved areas, and turning preservation on or off makes almost no difference. The cause is that the metric looks at a fixed small neighborhood around each vertex. On a dense mesh that neighborhood covers a tiny patch of surface, and adjacent triangles end up nearly flat relative to each other even when the underlying surface is clearly curving. The fix would be to grow the

sampling region based on the mesh's edge length, but implementing that would undo the empirical observation, which is itself a useful result.

I also tried a performance optimization that backfired. My expectation was that swapping the standard tree-based map for a hash-based one would cut 20 to 40 percent off the collapse loop. Measured on the dragon model, it made things 75 percent slower at aggressive targets and roughly the same at moderate ones. The reason is that the collapse loop touches keys in a spatially local pattern that happens to suit tree descents, while the hash map scatters those keys across buckets and loses the locality. I reverted the change and wrote up the result, because it contradicts the common assumption that hash maps are always faster.

The project started as a command-line experiment inside a larger engine and ended as a standalone tool that makes both the algorithm and its limitations visible in real time. The parts I'm proudest of are the ones that took the longest to get honest about, the curvature limitation and the failed optimization. Both are documented with measurements and concrete next steps, and both helped me realize that characterizing what doesn't work is often more valuable than shipping another feature.